```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader

# ========================
# 1. CONFIG & DEVICE
# ========================
batch_size = 128
epochs = 30
learning_rate = 0.01

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Training on: {device}")

# For reproducibility (optional)
torch.manual_seed(42)
if device.type == "cuda":
    torch.cuda.manual_seed_all(42)

# ========================
# 2. DATASET & DATALOADERS
# ========================

# FashionMNIST: 28x28 grayscale images, 10 classes
# We'll use simple augmentation on train set to help generalization
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomCrop(28, padding=2),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # grayscale mean/std
])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

data_root = "~/.pytorch/F_MNIST_data/"

print("Loading FashionMNIST...")
train_dataset = datasets.FashionMNIST(
    root=data_root,
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = datasets.FashionMNIST(
    root=data_root,
    train=False,
    download=True,
    transform=test_transform
)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2, pin_memory=
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2, pin_memory=T

class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

# ------------------------
```

```python
# ==========================
# 3. MODEL DEFINITION
# ==========================

def get_fashion_resnet(num_classes: int = 10):
    """
    ResNet-18 adapted for FashionMNIST (1-channel, 28x28 images).
    """
    model = models.resnet18(weights=None)  # start from scratch for simplicity

    # Change input conv layer to accept 1-channel (grayscale)
    model.conv1 = nn.Conv2d(
        in_channels=1,
        out_channels=64,
        kernel_size=3,
        stride=1,
        padding=1,
        bias=False
    )

    # Remove the large initial maxpool — images are small (28x28)
    model.maxpool = nn.Identity()

    # Change the final fully connected layer to output num_classes logits
    model.fc = nn.Linear(in_features=512, out_features=num_classes)

    return model

model = get_fashion_resnet(num_classes=len(class_names))
model = model.to(device)

# ==========================
# 4. LOSS, OPTIMIZER, SCHEDULER
# ==========================
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

optimizer = optim.AdamW(
    model.parameters(),
    lr=learning_rate,
    weight_decay=1e-4
)

scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=epochs
)
```

```
Training on: cuda
Loading FashionMNIST...
```

```python
# ==========================
# 5. TRAIN & EVAL FUNCTIONS
# ==========================

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0
    total = 0
    correct = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        _, preds = torch.max(outputs, dim=1)
        correct += (preds == labels).sum().item()
```

```
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = 100.0 * correct / total
    return epoch_loss, epoch_acc


def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    total = 0
    correct = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)

            _, preds = torch.max(outputs, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = 100.0 * correct / total
    return epoch_loss, epoch_acc
```

```
# =========================
# 6. TRAINING LOOP
# =========================

best_acc = 0.0
best_model_path = "fashion_resnet_best.pth"

print(f"Starting classification training for {epochs} epochs...")

for epoch in range(epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(model, test_loader, criterion, device)

    scheduler.step()
    current_lr = optimizer.param_groups[0]['lr']

    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), best_model_path)

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"| LR: {current_lr:.5f} "
        f"| Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}% "
        f"| Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}% "
        f"(Best Val Acc: {best_acc:.2f}%)"
    )

print("\nTraining complete.")
print(f"Best validation accuracy: {best_acc:.2f}%")
print(f"Best model saved to: {best_model_path}")
```

```
Starting classification training for 30 epochs...
Epoch [1/30] | LR: 0.00997 | Train Loss: 1.0932, Train Acc: 74.07% | Val Loss: 1.6411, Val Acc: 48.00% (
Epoch [2/30] | LR: 0.00989 | Train Loss: 0.8044, Train Acc: 86.43% | Val Loss: 0.8067, Val Acc: 86.11% (
Epoch [3/30] | LR: 0.00976 | Train Loss: 0.7471, Train Acc: 89.13% | Val Loss: 0.7385, Val Acc: 88.98% (
Epoch [4/30] | LR: 0.00957 | Train Loss: 0.7168, Train Acc: 90.29% | Val Loss: 0.7492, Val Acc: 89.40% (
Epoch [5/30] | LR: 0.00933 | Train Loss: 0.6998, Train Acc: 91.12% | Val Loss: 0.6988, Val Acc: 90.68% (
Epoch [6/30] | LR: 0.00905 | Train Loss: 0.6889, Train Acc: 91.61% | Val Loss: 0.7029, Val Acc: 90.62% (
Epoch [7/30] | LR: 0.00872 | Train Loss: 0.6739, Train Acc: 92.30% | Val Loss: 0.6854, Val Acc: 91.62% (
Epoch [8/30] | LR: 0.00835 | Train Loss: 0.6640, Train Acc: 92.67% | Val Loss: 0.6777, Val Acc: 92.17% (
Epoch [9/30] | LR: 0.00794 | Train Loss: 0.6558, Train Acc: 93.11% | Val Loss: 0.6718, Val Acc: 92.04% (
Epoch [10/30] | LR: 0.00750 | Train Loss: 0.6502, Train Acc: 93.29% | Val Loss: 0.6675, Val Acc: 92.44%
```

```
Epoch [11/30] | LR: 0.00703 | Train Loss: 0.6398, Train Acc: 93.78% | Val Loss: 0.6537, Val Acc: 93.19%
Epoch [12/30] | LR: 0.00655 | Train Loss: 0.6348, Train Acc: 94.01% | Val Loss: 0.6496, Val Acc: 93.32%
Epoch [13/30] | LR: 0.00604 | Train Loss: 0.6263, Train Acc: 94.45% | Val Loss: 0.6472, Val Acc: 93.43%
Epoch [14/30] | LR: 0.00552 | Train Loss: 0.6186, Train Acc: 94.75% | Val Loss: 0.6763, Val Acc: 92.23%
Epoch [15/30] | LR: 0.00500 | Train Loss: 0.6142, Train Acc: 94.90% | Val Loss: 0.6404, Val Acc: 93.86%
Epoch [16/30] | LR: 0.00448 | Train Loss: 0.6053, Train Acc: 95.33% | Val Loss: 0.6481, Val Acc: 93.46%
Epoch [17/30] | LR: 0.00396 | Train Loss: 0.5996, Train Acc: 95.57% | Val Loss: 0.6288, Val Acc: 94.31%
Epoch [18/30] | LR: 0.00345 | Train Loss: 0.5922, Train Acc: 95.93% | Val Loss: 0.6365, Val Acc: 94.12%
Epoch [19/30] | LR: 0.00297 | Train Loss: 0.5867, Train Acc: 96.19% | Val Loss: 0.6255, Val Acc: 94.60%
Epoch [20/30] | LR: 0.00250 | Train Loss: 0.5795, Train Acc: 96.55% | Val Loss: 0.6262, Val Acc: 94.36%
Epoch [21/30] | LR: 0.00206 | Train Loss: 0.5728, Train Acc: 96.80% | Val Loss: 0.6288, Val Acc: 94.58%
Epoch [22/30] | LR: 0.00165 | Train Loss: 0.5667, Train Acc: 97.08% | Val Loss: 0.6243, Val Acc: 94.76%
Epoch [23/30] | LR: 0.00128 | Train Loss: 0.5606, Train Acc: 97.39% | Val Loss: 0.6216, Val Acc: 95.04%
Epoch [24/30] | LR: 0.00095 | Train Loss: 0.5565, Train Acc: 97.59% | Val Loss: 0.6227, Val Acc: 94.93%
Epoch [25/30] | LR: 0.00067 | Train Loss: 0.5504, Train Acc: 97.90% | Val Loss: 0.6206, Val Acc: 95.22%
Epoch [26/30] | LR: 0.00043 | Train Loss: 0.5471, Train Acc: 98.02% | Val Loss: 0.6201, Val Acc: 95.08%
Epoch [27/30] | LR: 0.00024 | Train Loss: 0.5430, Train Acc: 98.27% | Val Loss: 0.6199, Val Acc: 95.17%
Epoch [28/30] | LR: 0.00011 | Train Loss: 0.5408, Train Acc: 98.37% | Val Loss: 0.6200, Val Acc: 95.25%
Epoch [29/30] | LR: 0.00003 | Train Loss: 0.5385, Train Acc: 98.53% | Val Loss: 0.6206, Val Acc: 95.20%
Epoch [30/30] | LR: 0.00000 | Train Loss: 0.5363, Train Acc: 98.67% | Val Loss: 0.6202, Val Acc: 95.27%

Training complete.
Best validation accuracy: 95.27%
Best model saved to: fashion_resnet_best.pth
```

```
#takes 25 mins to run ^
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Price Prediction

```
import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.pipeline import Pipeline
```

```
DATA_PATH = Path("/content/transactions_articles_sample.csv")

df = pd.read_csv(DATA_PATH)

print("Original shape:", df.shape)
print(df.head().T)  # transpose to inspect columns quickly
```

```
colour_group_name                                                Dark Blue
```

```
colour_group_name                                                  Dark Blue
perceived_colour_value_id                                                   2
perceived_colour_value_name                                       Medium Dusty
perceived_colour_master_id                                                  2
perceived_colour_master_name                                             Blue
department_no                                                            8558
department_name                                                Men Sport Tops
index_code                                                                  S
index_name                                                             Sport
index_group_no                                                             26
index_group_name                                                       Sport
section_no                                                                 22
section_name                                                    Men H&M Sport
garment_group_no                                                        1007
garment_group_name                                                    Outdoor
detail_desc                   Jersey sports top with a zip at the top and st...

                                                                            4
t_dat                                                             2018-09-28
customer_id                   e86cfe736ab46f06691768e7a87ede453bca1b523531d7...
article_id                                                         621381011
price                                                               0.033881
sales_channel_id                                                           2
product_code                                                          621381
prod_name                                                  Jade Denim Trash TRS
product_type_no                                                         272
product_type_name                                                    Trousers
product_group_name                                       Garment Lower body
graphical_appearance_no                                               1010016
graphical_appearance_name                                               Solid
colour_group_code                                                           9
colour_group_name                                                       Black
perceived_colour_value_id                                                   4
perceived_colour_value_name                                              Dark
perceived_colour_master_id                                                  5
perceived_colour_master_name                                            Black
department_no                                                            1747
department_name                                                      Trousers
index_code                                                                  D
index_name                                                           Divided
index_group_no                                                              2
index_group_name                                                     Divided
section_no                                                                 53
section_name                                                Divided Collection
garment_group_no                                                        1009
garment_group_name                                                   Trousers
detail_desc                   High-waisted jeans in washed superstretch deni...
```

```
df["price_real_sek"] = df["price"] * 10000
df["price_usd"] = df["price_real_sek"] / 10.5   # approx SEK→USD conversion

print(df[["price", "price_real_sek", "price_usd"]].head())
```

```
      price  price_real_sek  price_usd
0  0.013542      135.423729  12.897498
1  0.011847      118.474576  11.283293
2  0.025407      254.067797  24.196933
3  0.032186      321.864407  30.653753
4  0.033881      338.813559  32.267958
```

```
cols_to_drop = [
    "t_dat", "customer_id", "article_id", "product_code",
    "prod_name", "detail_desc", "price", "price_real_sek"  # we now only use price_usd
]

df = df.drop(columns=[c for c in cols_to_drop if c in df.columns])

print("After dropping unnecessary columns:", df.shape)

# Drop rows missing our target
df = df.dropna(subset=["price_usd"])

# Optional: downsample for RAM
MAX_ROWS = 500_000
if len(df) > MAX_ROWS:
    df = df.sample(MAX_ROWS, random_state=42).reset_index(drop=True)
```

```
After dropping unnecessary columns: (2000000, 23)
```

```
target_col = "price_usd"
y = df[target_col]
X = df.drop(columns=[target_col])

categorical_cols = [c for c in X.columns if X[c].dtype == "object"]
numeric_cols = [c for c in X.columns if c not in categorical_cols]

print("Categorical cols:", categorical_cols)
print("Numeric cols:", numeric_cols)
```

```
Categorical cols: ['product_type_name', 'product_group_name', 'graphical_appearance_name', 'colour_group
Numeric cols: ['sales_channel_id', 'product_type_no', 'graphical_appearance_no', 'colour_group_code', 'p
```

```
#preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_cols),
        ("num", "passthrough", numeric_cols)
    ]
)

regressor = HistGradientBoostingRegressor(
    max_depth=8,
    learning_rate=0.1,
    max_iter=200,
    min_samples_leaf=50,
    random_state=42
)

model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("regressor", regressor)
])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# ========================
# 7. TRAIN MODEL
# ========================
model.fit(X_train, y_train)

# ========================
# 8. EVALUATE
# ========================
y_pred = model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"RMSE (USD): {rmse:.2f}")
print(f"R² Score:   {r2:.4f}")
```

```
RMSE (USD): 10.96
R² Score:   0.7499
```

```
import joblib

joblib.dump(model, "hm_price_regressor.pkl")
print("Saved regressor to hm_price_regressor.pkl")
```

```
Saved regressor to hm_price_regressor.pkl
```

```
# Assuming df is the same dataframe used for training the regressor
price_by_type = df.groupby("product_type_name")["price_usd"].median().sort_values()
print(price_by_type)
```

```
product_type_name
Hair string           3.890234
```

```
Hair clip             4.019370
Earrings              4.826473
Chem. cosmetics       4.826473
Hair ties             4.826473
                       ...
Weekend/Gym bag      54.866828
Boots                56.481033
Jacket               64.552058
Outdoor Waistcoat    64.552058
Coat                 96.836158
Name: price_usd, Length: 101, dtype: float64
```

```
# price_by_type you already have:
price_by_type = df.groupby("product_type_name")["price_usd"].median().sort_values()

print("Global median:", df["price_usd"].median())
print(price_by_type.tail(10))  # just to inspect the most expensive ones
```

```
Global median: 24.196933010492284
product_type_name
Underdress           48.410008
Blazer               48.410008
Outdoor trousers     48.410008
Outdoor overall      50.831316
Giftbox              53.656174
Weekend/Gym bag      54.866828
Boots                56.481033
Jacket               64.552058
Outdoor Waistcoat    64.552058
Coat                 96.836158
Name: price_usd, dtype: float64
```

```
def median_for_keywords(keywords):
    """
    keywords: list of strings like ["t-shirt", "top"]
    Returns median price_usd over all rows whose product_type_name
    contains ANY of those keywords (case-insensitive).
    Falls back to global median if nothing matches.
    """
    mask = False
    for kw in keywords:
        mask = mask | df["product_type_name"].str.contains(kw, case=False, na=False)
    if mask.sum() == 0:
        return float(df["price_usd"].median())
    return float(df.loc[mask, "price_usd"].median())
```

```
class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

class_to_median_price = {
    "T-shirt/top": median_for_keywords(["t-shirt", "top", "tee"]),
    "Trouser":     median_for_keywords(["trouser", "pants", "jeans"]),
    "Pullover":    median_for_keywords(["pullover", "sweater", "knit"]),
    "Dress":       median_for_keywords(["dress"]),
    "Coat":        median_for_keywords(["coat", "jacket", "waistcoat"]),
    "Sandal":      median_for_keywords(["sandal"]),
    "Shirt":       median_for_keywords(["shirt", "blouse"]),
    "Sneaker":     median_for_keywords(["sneaker", "trainer", "shoe"]),
    "Bag":         median_for_keywords(["bag", "backpack", "pouch"]),
    "Ankle boot":  median_for_keywords(["boot"])
}

print("Class → median price (USD):")
```

```
for k, v in class_to_median_price.items():
    print(f"{k:12s}: ${v:.2f}")
```

```
Class → median price (USD):
T-shirt/top : $12.90
Trouser     : $32.27
Pullover    : $25.81
Dress       : $32.27
Coat        : $64.55
Sandal      : $19.35
Shirt       : $19.35
Sneaker     : $32.27
Bag         : $24.20
Ankle boot  : $56.48
```

```
def estimate_price_from_image(image_path: str):
    _, pred_class_name, probs = predict_class_from_image(image_path)
    base_price = class_to_median_price.get(pred_class_name, df["price_usd"].median())

    print(f"Predicted clothing class: {pred_class_name}")
    print(f"Estimated price (median for this class): ${base_price:.2f}")
    return base_price
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader
from PIL import Image
import numpy as np

# ========================
# 1. CONFIG & DEVICE
# ========================
batch_size = 128
epochs = 30
learning_rate = 0.01
weight_decay = 1e-4  # FIX: This was missing!

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Training on: {device}")

torch.manual_seed(42)
if device.type == "cuda":
    torch.cuda.manual_seed_all(42)

# ========================
# 2. DATASET & DATALOADERS
# ========================

train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomCrop(28, padding=2),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
```

```
])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

data_root = "~/.pytorch/F_MNIST_data/"

print("Loading FashionMNIST...")
train_dataset = datasets.FashionMNIST(
    root=data_root,
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = datasets.FashionMNIST(
    root=data_root,
    train=False,
    download=True,
    transform=test_transform
)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2, pin_memory
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2, pin_memory=

class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

# ==========================
# 3. MODEL DEFINITION
# ==========================

def get_fashion_resnet(num_classes: int = 10):
    """
    For FashionMNIST (grayscale): train from scratch with 1-channel input
    """
    model = models.resnet18(weights=None)
    model.conv1 = nn.Conv2d(
        in_channels=1,
        out_channels=64,
        kernel_size=3,
        stride=1,
        padding=1,
        bias=False
    )
    model.maxpool = nn.Identity()
    model.fc = nn.Linear(in_features=512, out_features=num_classes)
    return model

def get_pretrained_fashion_resnet(num_classes: int = 10):
    """
    For real RGB images: use pre-trained weights and fine-tune
    """
    model = models.resnet18(weights='IMAGENET1K_V1')  # Pre-trained on RGB

    # Keep the RGB input (3 channels) - don't modify conv1
    # Only replace the final layer
    model.fc = nn.Linear(in_features=512, out_features=num_classes)

    return model

# Use the grayscale version for FashionMNIST training
```

```python
model = get_fashion_resnet(num_classes=len(class_names))
model = model.to(device)

# ========================
# 4. LOSS, OPTIMIZER, SCHEDULER
# ========================
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

optimizer = optim.AdamW(
    model.parameters(),
    lr=learning_rate,
    weight_decay=weight_decay  # Now defined
)

scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=epochs
)

# ========================
# 5. TRAIN & EVAL FUNCTIONS
# ========================

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0
    total = 0
    correct = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        _, preds = torch.max(outputs, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = 100.0 * correct / total
    return epoch_loss, epoch_acc


def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    total = 0
    correct = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)

            _, preds = torch.max(outputs, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = 100.0 * correct / total
    return epoch_loss, epoch_acc
```

```python
# ==========================
# 6. TRAINING LOOP
# ==========================

best_acc = 0.0
best_model_path = "fashion_resnet_best.pth"

print(f"Starting classification training for {epochs} epochs...")

for epoch in range(epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(model, test_loader, criterion, device)

    scheduler.step()
    current_lr = optimizer.param_groups[0]['lr']

    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), best_model_path)

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"| LR: {current_lr:.5f} "
        f"| Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}% "
        f"| Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}% "
        f"(Best Val Acc: {best_acc:.2f}%)"
    )

print("\nTraining complete.")
print(f"Best validation accuracy: {best_acc:.2f}%")
print(f"Best model saved to: {best_model_path}")
```

```
Training on: cuda
Loading FashionMNIST...
100%|██████████| 26.4M/26.4M [00:02<00:00, 10.8MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 192kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.50MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 15.5MB/s]
Starting classification training for 30 epochs...
Epoch [1/30] | LR: 0.00997 | Train Loss: 1.0858, Train Acc: 74.38% | Val Loss: 2.3735, Val Acc: 35.10% (
Epoch [2/30] | LR: 0.00989 | Train Loss: 0.7977, Train Acc: 86.80% | Val Loss: 0.7838, Val Acc: 86.67% (
Epoch [3/30] | LR: 0.00976 | Train Loss: 0.7439, Train Acc: 89.27% | Val Loss: 0.7668, Val Acc: 88.19% (
Epoch [4/30] | LR: 0.00957 | Train Loss: 0.7173, Train Acc: 90.28% | Val Loss: 0.7185, Val Acc: 90.54% (
Epoch [5/30] | LR: 0.00933 | Train Loss: 0.6994, Train Acc: 91.17% | Val Loss: 0.6873, Val Acc: 91.31% (
Epoch [6/30] | LR: 0.00905 | Train Loss: 0.6893, Train Acc: 91.60% | Val Loss: 0.6921, Val Acc: 91.15% (
Epoch [7/30] | LR: 0.00872 | Train Loss: 0.6738, Train Acc: 92.32% | Val Loss: 0.6859, Val Acc: 91.65% (
Epoch [8/30] | LR: 0.00835 | Train Loss: 0.6644, Train Acc: 92.72% | Val Loss: 0.6962, Val Acc: 91.21% (
Epoch [9/30] | LR: 0.00794 | Train Loss: 0.6554, Train Acc: 93.04% | Val Loss: 0.6781, Val Acc: 91.72% (
Epoch [10/30] | LR: 0.00750 | Train Loss: 0.6504, Train Acc: 93.31% | Val Loss: 0.6849, Val Acc: 91.65%
Epoch [11/30] | LR: 0.00703 | Train Loss: 0.6421, Train Acc: 93.67% | Val Loss: 0.6549, Val Acc: 93.08%
Epoch [12/30] | LR: 0.00655 | Train Loss: 0.6343, Train Acc: 94.03% | Val Loss: 0.6501, Val Acc: 93.38%
Epoch [13/30] | LR: 0.00604 | Train Loss: 0.6257, Train Acc: 94.42% | Val Loss: 0.6474, Val Acc: 93.11%
Epoch [14/30] | LR: 0.00552 | Train Loss: 0.6185, Train Acc: 94.77% | Val Loss: 0.6608, Val Acc: 92.83%
Epoch [15/30] | LR: 0.00500 | Train Loss: 0.6143, Train Acc: 94.90% | Val Loss: 0.6425, Val Acc: 93.56%
Epoch [16/30] | LR: 0.00448 | Train Loss: 0.6055, Train Acc: 95.35% | Val Loss: 0.6395, Val Acc: 93.57%
Epoch [17/30] | LR: 0.00396 | Train Loss: 0.6004, Train Acc: 95.57% | Val Loss: 0.6280, Val Acc: 93.95%
Epoch [18/30] | LR: 0.00345 | Train Loss: 0.5925, Train Acc: 95.92% | Val Loss: 0.6296, Val Acc: 94.26%
Epoch [19/30] | LR: 0.00297 | Train Loss: 0.5871, Train Acc: 96.20% | Val Loss: 0.6311, Val Acc: 94.26%
Epoch [20/30] | LR: 0.00250 | Train Loss: 0.5789, Train Acc: 96.50% | Val Loss: 0.6323, Val Acc: 94.22%
Epoch [21/30] | LR: 0.00206 | Train Loss: 0.5726, Train Acc: 96.78% | Val Loss: 0.6242, Val Acc: 94.56%
Epoch [22/30] | LR: 0.00165 | Train Loss: 0.5668, Train Acc: 97.10% | Val Loss: 0.6245, Val Acc: 94.83%
Epoch [23/30] | LR: 0.00128 | Train Loss: 0.5610, Train Acc: 97.38% | Val Loss: 0.6243, Val Acc: 94.79%
Epoch [24/30] | LR: 0.00095 | Train Loss: 0.5564, Train Acc: 97.58% | Val Loss: 0.6230, Val Acc: 94.84%
Epoch [25/30] | LR: 0.00067 | Train Loss: 0.5508, Train Acc: 97.84% | Val Loss: 0.6194, Val Acc: 95.11%
Epoch [26/30] | LR: 0.00043 | Train Loss: 0.5457, Train Acc: 98.14% | Val Loss: 0.6225, Val Acc: 94.93%
Epoch [27/30] | LR: 0.00024 | Train Loss: 0.5440, Train Acc: 98.19% | Val Loss: 0.6218, Val Acc: 95.13%
Epoch [28/30] | LR: 0.00011 | Train Loss: 0.5407, Train Acc: 98.38% | Val Loss: 0.6217, Val Acc: 95.17%
Epoch [29/30] | LR: 0.00003 | Train Loss: 0.5381, Train Acc: 98.51% | Val Loss: 0.6221, Val Acc: 95.15%
Epoch [30/30] | LR: 0.00000 | Train Loss: 0.5372, Train Acc: 98.53% | Val Loss: 0.6219, Val Acc: 95.13%

Training complete.
Best validation accuracy: 95.17%
Best model saved to: fashion_resnet_best.pth
```

```python
!pip install git+https://github.com/openai/CLIP.git
```

```python
import torch
import clip
from PIL import Image

# 1. LOAD CLIP MODEL

device = "cuda" if torch.cuda.is_available() else "cpu"
model, preprocess = clip.load("ViT-B/32", device=device)

print(f"Using device: {device}")
print("CLIP model loaded successfully!")

# 2. DEFINE CATEGORIES

class_names = [
    "T-shirt",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

class_to_median_price = {
    "T-shirt": 12.90,
    "Trouser": 32.27,
    "Pullover": 25.81,
    "Dress": 32.27,
    "Coat": 64.55,
    "Sandal": 19.35,
    "Shirt": 19.35,
    "Sneaker": 32.27,
    "Bag": 24.20,
    "Ankle boot": 56.48,
}

# Create text prompts (better descriptions improve accuracy)
text_prompts = [
    "a photo of a t-shirt",
    "a photo of trousers",
    "a photo of a pullover sweater",
    "a photo of a dress",
    "a photo of a coat",
    "a photo of sandals",
    "a photo of a shirt",
    "a photo of sneakers",
    "a photo of a bag",
    "a photo of ankle boots"
]

# Encode text prompts
text_tokens = clip.tokenize(text_prompts).to(device)
with torch.no_grad():
    text_features = model.encode_text(text_tokens)
    text_features /= text_features.norm(dim=-1, keepdim=True)

# 3. PREDICTION FUNCTION

def predict_clothing_clip(image_path: str):
    """
    Predict clothing class using CLIP (zero-shot)
    Works on real images without any training!
    """
    # Load and preprocess image
    image = preprocess(Image.open(image_path)).unsqueeze(0).to(device)

    # Get image features
    with torch.no_grad():
        image_features = model.encode_image(image)
        image_features /= image_features.norm(dim=-1, keepdim=True)
```

```
        # Calculate similarity scores
        similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)
        values, indices = similarity[0].topk(5)

    # Get prediction
    pred_idx = indices[0].item()
    pred_class = class_names[pred_idx]
    confidence = values[0].item()

    # Get price
    price = class_to_median_price.get(pred_class, 24.20)

    # Display results
    print(f"\n{'='*60}")
    print(f"Image: {image_path}")
    print(f"Model: CLIP (Zero-Shot, No Training)")
    print(f"{'='*60}")
    print(f"Predicted: {pred_class.upper()}")
    print(f"Confidence: {confidence*100:.2f}%")
    print(f"\nTop-5 predictions:")
    for i in range(5):
        idx = indices[i].item()
        prob = values[i].item()
        print(f"  {class_names[idx]:15s}: {prob*100:.2f}%")
    print(f"\nEstimated price: ${price:.2f} USD")
    print(f"{'='*60}\n")

    return pred_class, price

# 4. USAGE

# Test on your images - NO TRAINING NEEDED!
predict_clothing_clip("/content/coat.png")
predict_clothing_clip("/content/shirt.png")
predict_clothing_clip("/content/ldress.jpg")

# This should give MUCH better results on real photos!
```

```
============================================================
Image: /content/ldress.jpg
Model: CLIP (Zero-Shot, No Training)
============================================================
Predicted: DRESS
Confidence: 99.27%

Top-5 predictions:
  Dress          : 99.27%
  Shirt          : 0.36%
  Coat           : 0.24%
  T-shirt        : 0.04%
  Bag            : 0.03%

Estimated price: $32.27 USD
============================================================

('Dress', 32.27)
```

Start coding or generate with AI.

```
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lear
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from sci
```

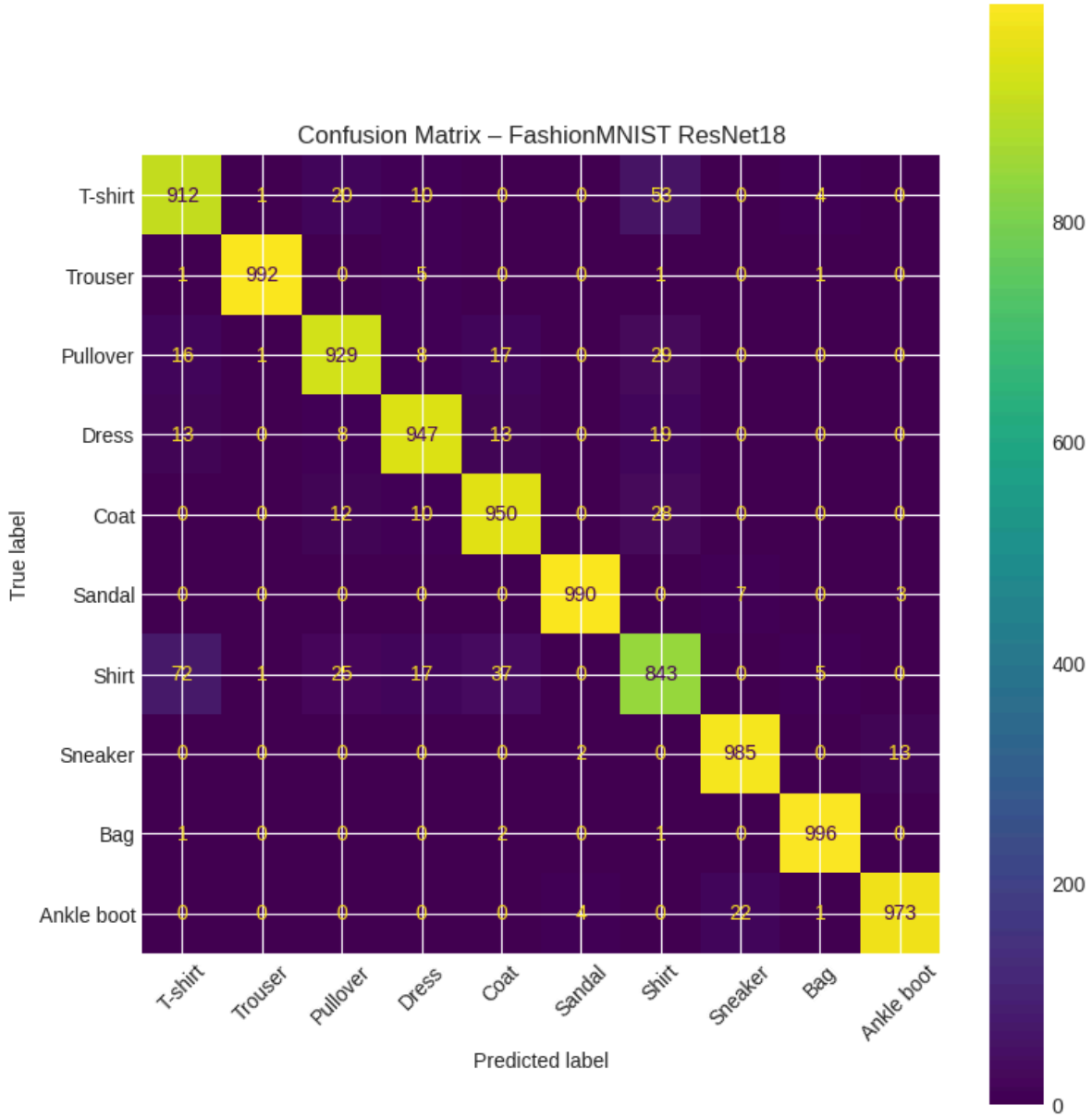
```
import torch
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Recreate the same model architecture used for training
fashion_model = get_fashion_resnet(num_classes=len(class_names))
fashion_model.load_state_dict(torch.load("fashion_resnet_best.pth", map_location=device))
fashion_model.to(device)
fashion_model.eval()
```

```
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
```

```
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=10, bias=True)
)
```

```
all_labels = []
all_preds = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = fashion_model(images)
        _, preds = torch.max(outputs, dim=1)

        all_labels.append(labels.cpu().numpy())
        all_preds.append(preds.cpu().numpy())

all_labels = np.concatenate(all_labels)
all_preds = np.concatenate(all_preds)
```

```
cm = confusion_matrix(all_labels, all_preds)

fig, ax = plt.subplots(figsize=(8, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(ax=ax, xticks_rotation=45)
plt.title("Confusion Matrix — FashionMNIST ResNet18")
plt.tight_layout()
plt.show()
```



Start coding or generate with AI.

Start coding or generate with AI.

```
import matplotlib.pyplot as plt

# 1. Reconstruct history from your epoch prints

train_losses = [
    1.0858,
    0.7977,
    0.7439,
    0.7173,
    0.6994,
    0.6893,
    0.6738,
    0.6644,
    0.6554,
    0.6504,
    0.6421,
    0.6343,
    0.6257,
    0.6185,
    0.6143,
    0.6055,
    0.6004,
    0.5925,
    0.5871,
    0.5789,
    0.5726,
```

```
    0.5668,
    0.5610,
    0.5564,
    0.5508,
    0.5457,
    0.5440,
    0.5407,
    0.5381,
    0.5372,
]

val_losses = [
    2.3735,
    0.7838,
    0.7668,
    0.7185,
    0.6873,
    0.6921,
    0.6859,
    0.6962,
    0.6781,
    0.6849,
    0.6549,
    0.6501,
    0.6474,
    0.6608,
    0.6425,
    0.6395,
    0.6280,
    0.6296,
    0.6311,
    0.6323,
    0.6242,
    0.6245,
    0.6243,
    0.6230,
    0.6194,
    0.6225,
    0.6218,
    0.6217,
    0.6221,
    0.6219,
]

train_accuracies = [
    74.38,
    86.80,
    89.27,
    90.28,
    91.17,
    91.60,
    92.32,
    92.72,
    93.04,
    93.31,
    93.67,
    94.03,
    94.42,
    94.77,
    94.90,
    95.35,
    95.57,
    95.92,
    96.20,
    96.50,
    96.78,
    97.10,
    97.38,
    97.58,
    97.84,
    98.14,
    98.19,
    98.38,
    98.51,
    98.53,
```

```
]

val_accuracies = [
    35.10,
    86.67,
    88.19,
    90.54,
    91.31,
    91.15,
    91.65,
    91.21,
    91.72,
    91.65,
    93.08,
    93.38,
    93.11,
    92.83,
    93.56,
    93.57,
    93.95,
    94.26,
    94.26,
    94.22,
    94.56,
    94.83,
    94.79,
    94.84,
    95.11,
    94.93,
    95.13,
    95.17,
    95.15,
    95.13,
]

epochs_range = range(1, len(train_losses) + 1)

# 2. Loss curve
plt.figure()
plt.plot(epochs_range, train_losses, label="Train Loss")
plt.plot(epochs_range, val_losses, label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss (ResNet18 on FashionMNIST)")
plt.legend()
plt.grid(True)
plt.show()

# 3. Accuracy curve
plt.figure()
plt.plot(epochs_range, train_accuracies, label="Train Accuracy")
plt.plot(epochs_range, val_accuracies, label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Training vs Validation Accuracy (ResNet18 on FashionMNIST)")
plt.legend()
plt.grid(True)
plt.show()
```

Training vs Validation Loss (ResNet18 on FashionMNIST)
Train Loss
Val Loss
Loss
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0
5
10
15
20
25
30
Epoch